

Lab #1. Multiplication and Division in RISC-V Assembly

1. Introduction

This project is designed to help you become familiar with the basic operations and instructions involved in writing assembly programs in RISC-V. You will implement shift-and-add multiplication and shift-and-subtract division for unsigned numbers, **without using the dedicated multiplication or division instructions**. The algorithm flowcharts for 8-bit numbers are provided in Figures 1 and 2 and are explained step by step in the description below.

1.1 Shift-and-add multiplier

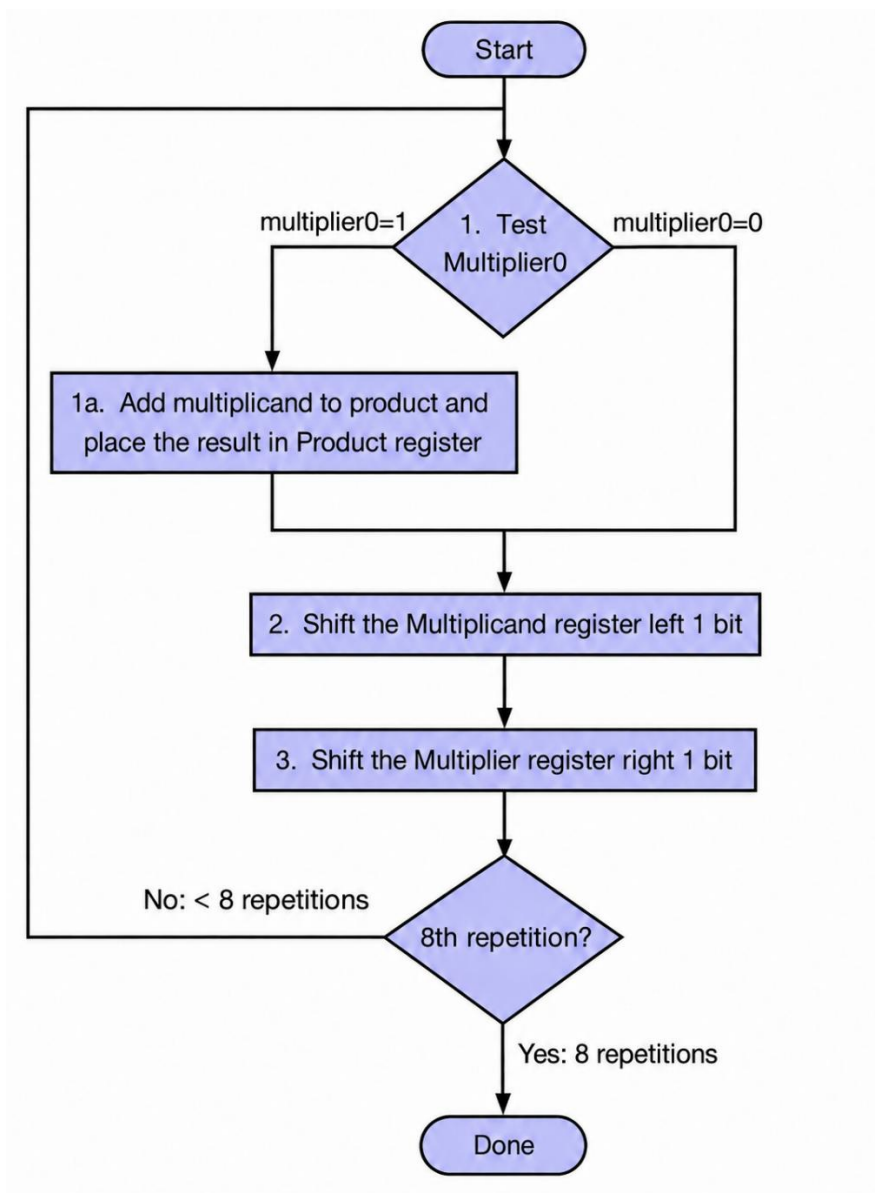


Figure 1. Flowchart for the baseline shift-and-add multiplication of 8-bit numbers.

The N -bit shift-and-add multiplication algorithm is similar to long multiplication and consists of the following steps:

0. Start with three registers:
 - Product register (initially zero, will hold the result)
 - Multiplier register (one number we are multiplying)
 - Multiplicand register (the other number we are multiplying)
1. Look at the least significant bit (rightmost bit) of the Multiplier
 - 1a. If it is 1, add the Multiplicand to the Product register
 - If it is 0, do nothing
2. Shift the Multiplicand register left by 1 bit
 - This is like multiplying the Multiplicand by 2, so it is ready for the next bit of the Multiplier
3. Shift the Multiplier register right by 1 bit
 - This moves the next Multiplier bit into the least significant position, so it can be tested in the next step.
4. Perform steps 1–3 a total of N times (for N -bit numbers)
 - Each repetition processes one bit of the Multiplier

Stop when all bits are processed

- At this point, the Product register contains the final multiplication result

1.2 Shift-and-subtract divider

The N -bit shift-and-subtract division algorithm is similar to long division and consists of the following steps:

0. Registers you need:
 - Remainder register (loaded initially with the dividend)
 - Divisor register (the number you're dividing by) - for baseline implementation, starts with divisor shifted by $N-1$ bit positions to the left, so that the divisor's bit 0 aligns with bit $N-1$ of the dividend register
 - Quotient register (starts at zero, will hold the result when done)
1. Subtract the Divisor from the Remainder and place the result into Remainder
2. Update the Quotient register:
 - 2a. If the subtraction succeeded ($\text{Remainder} \geq 0$):
 - Shift the Quotient left by 1, and set the new rightmost bit to 1
 - 2b. If the subtraction failed ($\text{Remainder} < 0$):

- If the result is negative, undo the subtraction - restore the Remainder by adding the Divisor and placing the result back into the Remainder
- Shift the Quotient left by 1, and set the new rightmost bit to 0.

3. Shift the Divisor register right by 1

- This lines it up with the next bit position, similar to how long division works when you “bring down” the next digit.

4. Perform steps 1–3 (subtract → test → update quotient → shift divisor) a total of N times

Stop when repetitions are done

- At the end:
 - The Quotient register holds the division result
 - The Remainder register holds the remainder (what’s left over)

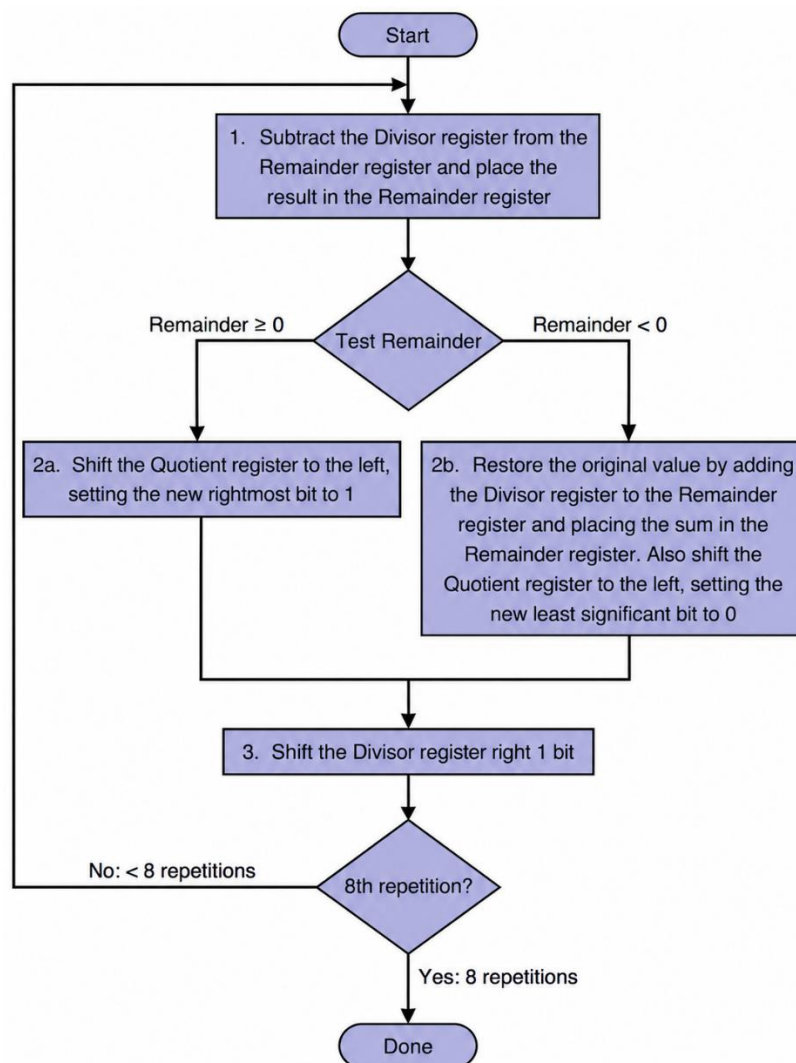


Figure 2. Flowchart for the baseline shift-and-subtract division of 8-bit numbers.

The baseline division algorithm uses $2N$ -bit-wide divisor and remainder registers and a $2N$ -bit adder/subtractor for N -bit inputs. In this lab, the inputs are at most 8 bits wide, so the available 32-bit registers and arithmetic operations provide sufficient width.

Both algorithms can be improved. For multiplication, the algorithm can stop early when all remaining bits of the multiplier are zero.

For division, let L and M denote the actual bit lengths of the dividend and divisor, respectively, excluding leading zeros. If the dividend is at least as large as the nonzero divisor, initially shift the divisor left by $L - M$ bit positions to align its most significant 1 bit with that of the dividend. Then perform $L - M + 1$ iterations, stopping after the iteration that uses the original, unshifted divisor. This can reduce the number of iterations compared with the baseline's fixed N iterations, at the cost of preprocessing to determine the initial alignment.

Handle special cases before performing this alignment: if the divisor is zero, use the division-by-zero behavior specified in the hints; otherwise, if the dividend is smaller than the divisor, return a quotient of zero and a remainder equal to the dividend.

To gain a better understanding of these algorithms, it is useful to work through an example on a piece of paper using smaller numbers.

Here is an example of baseline implementation of 4-bit division of the number 10 by 3. “R” is remainder. “D” is divisor. “Q” is quotient. “Before” = before steps 1-3 applied. “after” = after steps 1-3 applied. The final value for R (after) is 1.

Iter	R (before)	D (before) (divisor <=, uses 5 bits)	Subtract test	Quotient update	D (after)
1	1010	11000	< 0 → restore 1010	$Q \leftarrow (Q \ll 1) 0 \Rightarrow 0000$	01100
2	1010	01100	< 0 → restore 1010	$Q \leftarrow (Q \ll 1) 0 \Rightarrow 0000$	00110
3	1010	00110	= 0100 ≥ 0 (keep)	$Q \leftarrow (Q \ll 1) 1 \Rightarrow 0001$	00011
4	0100	00011	= 0001 ≥ 0 (keep)	$Q \leftarrow (Q \ll 1) 1 \Rightarrow 0011$	00001

Here is an example of more efficient implementation for the same numbers requiring only 3 steps.

Iter (bit i)	R (before)	D (before)	Subtract test	Outcome	Quotient bit	Q state	D after shift
2	1010	1100	negative → restore 10	fail	q2=0	0_ (---)	0110
1	1010	0110	= 0100 ≥ 0	keep	q1=1	01_ (partial)	0011
0	0100	0011	= 0001 ≥ 0	keep	q0=1	011 (final) 0001 → MSB shifts out	

Either implementation is acceptable in this lab.

2. Lab Tasks

The shift-and-add multiplier and shift-and-subtract divider can be implemented as a sequential circuits using a combination of shift registers and combinational logic. In this lab, we will implemented these algorithms as a code - an assembly programs - running on the RISC-V processor. Specifically, you will use base integer set of instructions (RV32I), e.g. any instruction from reference sheet page (Figure 3 below). All these instruction are covered in class. Avoid using pseudoinstruction that are not real instructions and translated to one or several real instructions upon assembly. The only exception is jump instruction “j label”, a pseudoinstruction that can be

used instead of real jump instruction “jal x0, label”. For a reference, see a list of pseudo-instruction (that should be avoided) in the textbook.

Note that you cannot implement multiplication by repeatedly (multiplier times) adding multiplicand. The same applies to division.

We will use RARS RISC-V emulator <https://github.com/TheThirdOne/rars> . RARS is distributed as an executable jar. You will need at least Java 8 to run it. For convenience the latest RARS executable file is included in the lab package. The last section in this document provides a mini tutorial on how to get started. RARS will be also covered on the first discussion sections.

Also provided are a skeletons of code for each code. You are to write the body of each function.

2.1 Multiplication

For the multiplication function, start with the skeleton code in mult.s. The two operands are provided in a0 (multiplicand) and a1 (multiplier). Both operands are unsigned integers of up to 8 bits. Store the product, which may require up to 16 bits, in a2.

$$a2 = a0 \times a1$$

2.2 Division

For the division function, start with the skeleton code in div.s. The 8-bit unsigned dividend is provided in a0, and the 8-bit unsigned divisor is provided in a1. Store the quotient in a2 and the remainder in a3; both fit within 8 bits.

$$a0 = a1 \times a2 + a3,$$

Here, a0 and a1 represent the original dividend and divisor. For a nonzero divisor, the remainder satisfies $0 \leq a3 < a1$.

Ensure that your code handles all relevant corner cases, including division by zero. If the divisor is zero, set all 8 bits of the quotient to 1 (0x000000FF) and set the remainder to the dividend.

3. Hints

- The code for each function can be greatly simplified through looping.
- For testing, you can edit op1 and op2 located at the top of the files. op1 is loaded into a0 and op2 is loaded into a1.
- After subtracting the shifted divisor from the remainder, check the intermediate result before restoring it. With this lab’s 8-bit inputs and 32-bit arithmetic, bit 31 (MSB) is 1 if the result is negative.

- You may use any temporary register t0-t6 and may scramble (do not need to preserve) argument registers a0-a1 for your code.

4. Lab Report

Your lab report should be a single PDF file, which has all the following items in the following order and clearly labeled:

1. **Please indicate how many hours you spent on this lab.** This will be helpful for calibrating the workload for the next time the course is taught.
2. Your “mult.s” assembly code *
3. Your “div.s” assembly code. *
4. If you have any feedback on how we might make the lab even better for next quarter, that’s always welcome. Please submit it in writing at the end of your lab

* For your code files, please copy and paste your code clearly in a monospace font (ex., Courier New). Provide in the comment the equivalent C code for each added line of the code in the skeleton.

5. Autograder

You will need to submit your “mult.s” and “div.s” files to the Gradescope assignment “lab1 Code”. When you make a submission, an autograder will run several tests to ensure your design is correct. You must pass all the tests to get a full score on the lab. You can resubmit as many times as you like.

6. What to Turn In

You will need to make two separate Gradescope submissions: “lab1 Code” and “lab1 Report”.

- For “lab1 Code”, please submit your “mult.s” and “div.s” files.
- For “lab1 Report”, please submit your lab report PDF file.

Only one submission per group is needed; but be sure to add all your group members in the report header and to the top of the code.

RISC-V Instruction Set Summary

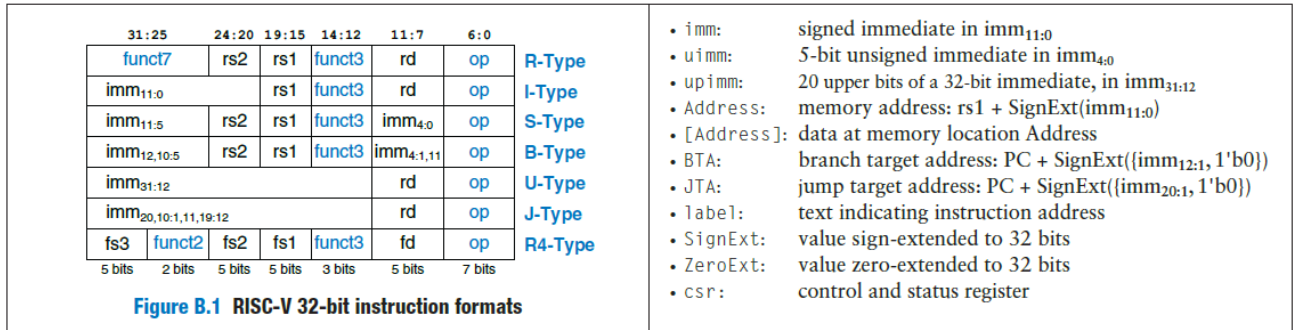


Table B.1 RV32I: RISC-V integer instructions

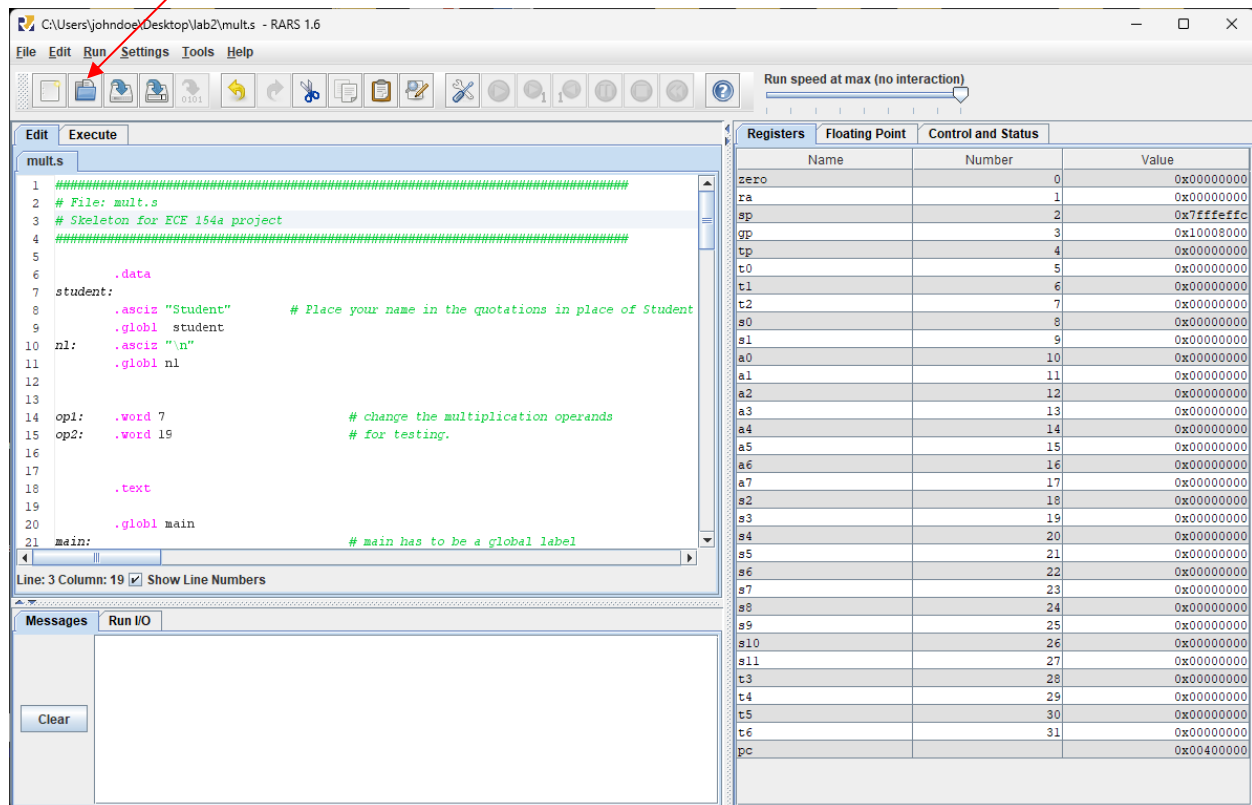
op	funct3	funct7	Type	Instruction	Description	Operation
0000011 (3)	000	–	I	lb rd, imm(rs1)	load byte	rd = SignExt([Address] _{7:0})
0000011 (3)	001	–	I	lh rd, imm(rs1)	load half	rd = SignExt([Address] _{15:0})
0000011 (3)	010	–	I	lw rd, imm(rs1)	load word	rd = [Address] _{31:0}
0000011 (3)	100	–	I	lbu rd, imm(rs1)	load byte unsigned	rd = ZeroExt([Address] _{7:0})
0000011 (3)	101	–	I	lhu rd, imm(rs1)	load half unsigned	rd = ZeroExt([Address] _{15:0})
0010011 (19)	000	–	I	addi rd, rs1, imm	add immediate	rd = rs1 + SignExt(imm)
0010011 (19)	001	0000000*	I	slli rd, rs1, uimm	shift left logical immediate	rd = rs1 << uimm
0010011 (19)	010	–	I	slti rd, rs1, imm	set less than immediate	rd = (rs1 < SignExt(imm))
0010011 (19)	011	–	I	sltiu rd, rs1, imm	set less than imm. unsigned	rd = (rs1 < SignExt(imm))
0010011 (19)	100	–	I	xori rd, rs1, imm	xor immediate	rd = rs1 ^ SignExt(imm)
0010011 (19)	101	0000000*	I	srlr rd, rs1, uimm	shift right logical immediate	rd = rs1 >> uimm
0010011 (19)	101	0100000*	I	srair rd, rs1, uimm	shift right arithmetic imm.	rd = rs1 >>> uimm
0010011 (19)	110	–	I	ori rd, rs1, imm	or immediate	rd = rs1 SignExt(imm)
0010011 (19)	111	–	I	andi rd, rs1, imm	and immediate	rd = rs1 & SignExt(imm)
0010111 (23)	–	–	U	auipc rd, upimm	add upper immediate to PC	rd = {upimm, 12'b0} + PC
0100011 (35)	000	–	S	sb rs2, imm(rs1)	store byte	[Address] _{7:0} = rs2 _{7:0}
0100011 (35)	001	–	S	sh rs2, imm(rs1)	store half	[Address] _{15:0} = rs2 _{15:0}
0100011 (35)	010	–	S	sw rs2, imm(rs1)	store word	[Address] _{31:0} = rs2
0110011 (51)	000	0000000	R	add rd, rs1, rs2	add	rd = rs1 + rs2
0110011 (51)	000	0100000	R	sub rd, rs1, rs2	sub	rd = rs1 – rs2
0110011 (51)	001	0000000	R	sll rd, rs1, rs2	shift left logical	rd = rs1 << rs2 _{4:0}
0110011 (51)	010	0000000	R	slt rd, rs1, rs2	set less than	rd = (rs1 < rs2)
0110011 (51)	011	0000000	R	sltu rd, rs1, rs2	set less than unsigned	rd = (rs1 < rs2)
0110011 (51)	100	0000000	R	xor rd, rs1, rs2	xor	rd = rs1 ^ rs2
0110011 (51)	101	0000000	R	srl rd, rs1, rs2	shift right logical	rd = rs1 >> rs2 _{4:0}
0110011 (51)	101	0100000	R	sra rd, rs1, rs2	shift right arithmetic	rd = rs1 >>> rs2 _{4:0}
0110011 (51)	110	0000000	R	or rd, rs1, rs2	or	rd = rs1 rs2
0110011 (51)	111	0000000	R	and rd, rs1, rs2	and	rd = rs1 & rs2
0110111 (55)	–	–	U	lui rd, upimm	load upper immediate	rd = {upimm, 12'b0}
1100011 (99)	000	–	B	beq rs1, rs2, label	branch if =	if (rs1 == rs2) PC = BTA
1100011 (99)	001	–	B	bne rs1, rs2, label	branch if ≠	if (rs1 ≠ rs2) PC = BTA
1100011 (99)	100	–	B	blt rs1, rs2, label	branch if <	if (rs1 < rs2) PC = BTA
1100011 (99)	101	–	B	bge rs1, rs2, label	branch if ≥	if (rs1 ≥ rs2) PC = BTA
1100011 (99)	110	–	B	bltu rs1, rs2, label	branch if < unsigned	if (rs1 < rs2) PC = BTA
1100011 (99)	111	–	B	bgeu rs1, rs2, label	branch if ≥ unsigned	if (rs1 ≥ rs2) PC = BTA
1100111 (103)	000	–	I	jalr rd, rs1, imm	jump and link register	PC = rs1 + SignExt(imm), rd = PC + 4
1101111 (111)	–	–	J	jal rd, label	jump and link	PC = JTA, rd = PC + 4

*Encoded in instr_{31:25}, the upper seven bits of the immediate field

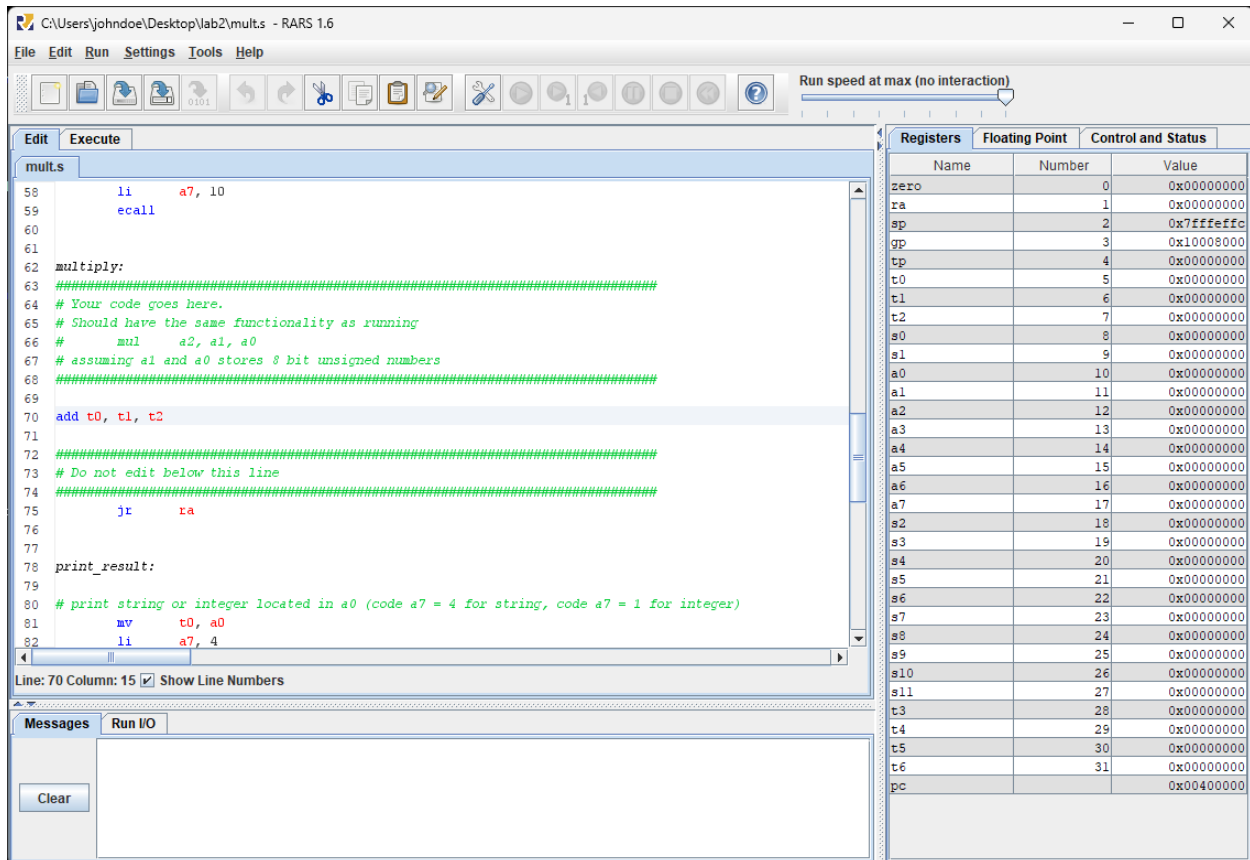
Figure 3. Set of instructions that can be used in this lab (copied from the textbook).

7. RARS mini tutorial

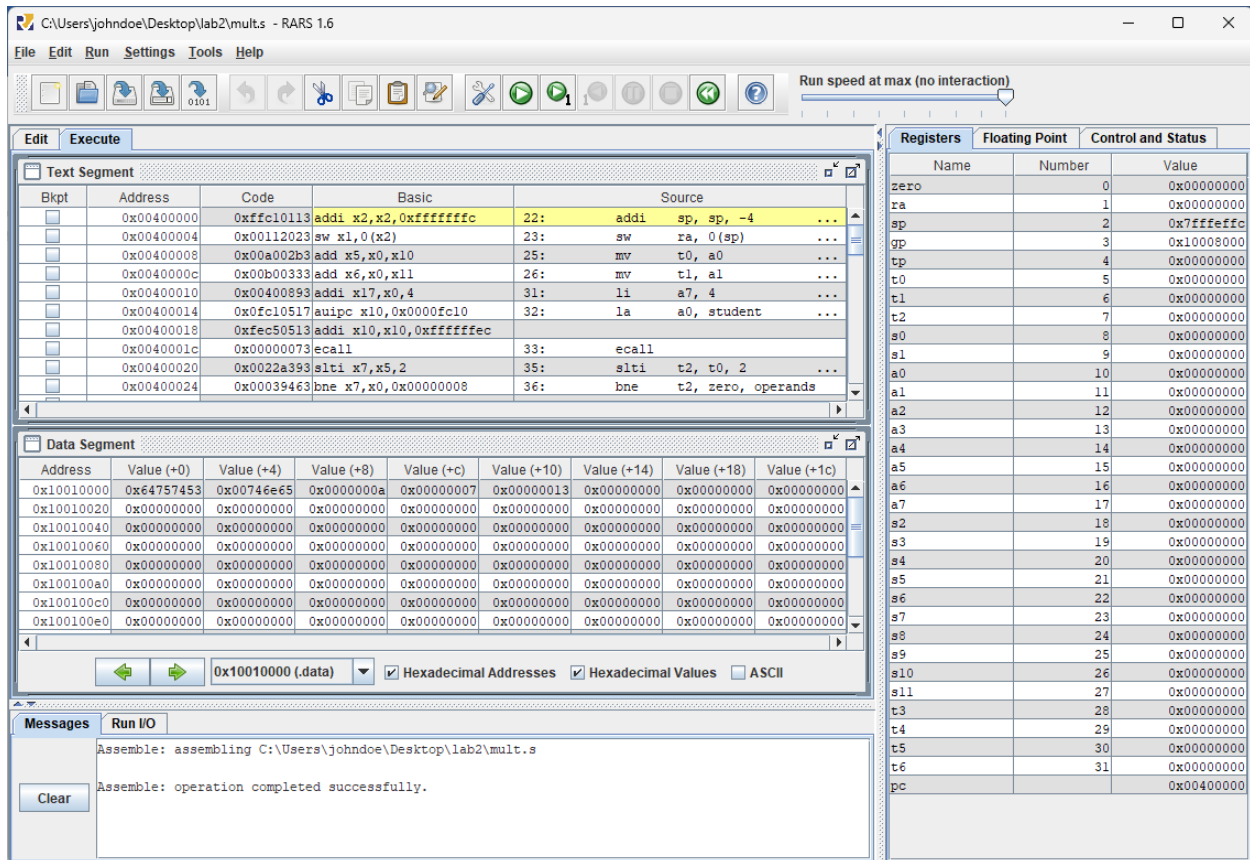
- 1) Open rars1_6.jar and then load mult.s file, both files provided in the lab package, by clicking on the open file button (pointed by arrow) or using menu. You will see the following content.



- 2) “Edit” window shows your assembly file
- 3) “Registers” window shows content of registers (“value”) in hexadecimal format with registers labeled by using both human-readable “name” as well as numeric x_% “number” format
- 4) In assembly file, put your name in line 8. Lines 14 and 15 are op1 and op2 values that are preloaded into registers a0 and a1, correspondingly. They can be changed if you need to verify correctness of your implementation with other arguments.
- 5) Scroll to the line below “# Your code goes below”. This is a body of a function where you need to put your code. In the skeleton file this portion is empty. As an example, in the screenshot below, instruction “add t0, t1, t2” is added.



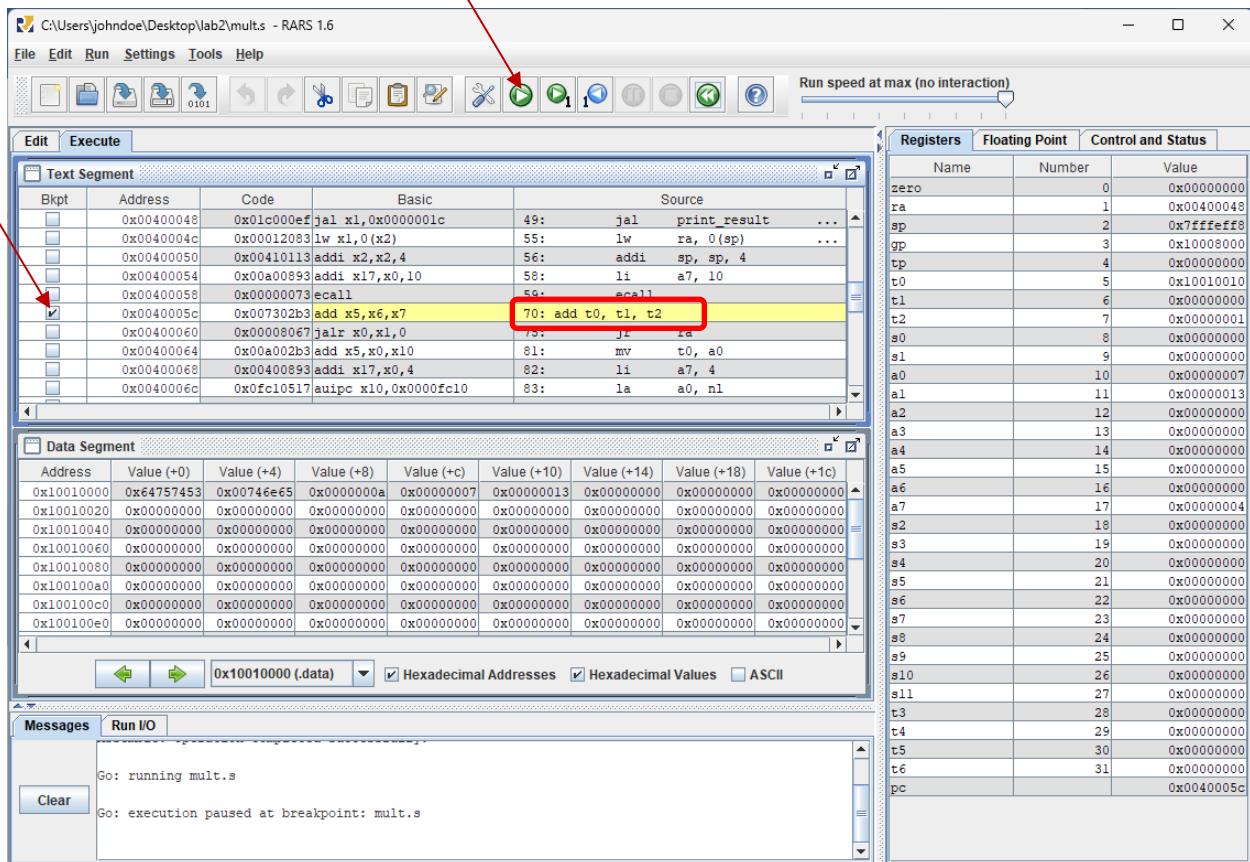
- 6) To test your code, first assemble your program by “Run”→ “Assembly”. This operation translates a set of assembly language instructions into the computer's lower-level machine code. In case of RARS, it also loads the code and data into memory, and initializes program counter to the address (0x400000) corresponding to the first instruction pointed by “main” label in the code. (More on program compilation and loading will be discussed later in class). If assemble operation is not successful, e.g., because incorrect use of assembly instructions, you will see some messages in the “message window”.
- 7) After successful assemble operation, RARS automatically switches to “execute” window to show “text segment” and “data” subwindows – see below. The former shows the code loaded in memory, with each line corresponding to one instruction (32 bit value in memory). Specifically, the location of 32-bit instruction in memory is shown in “Address” field, the machine code of the instruction is shown as hexadecimal number in “Code” field, the corresponding translated real instruction using x% registers is shown in “Basic” field, and a corresponding human-readable pseudo or real instruction is shown in the “Source” field. The latter corresponds to the original assembly program. The data segment shows the portion of a memory where data are loaded, with a content of a memory shown with multiple consecutive 32-bit hexadecimal values in each row, while their location in memory specified by the “Address” field.



- 8) When debugging your program you may use “run1 step at a time” button (white arrow in green circle with 1). The yellow background highlights the instruction that will be executed next.

Or you can use breakpoints. For example, find the first instruction in your multiplier code, check “Bkpt” field and then use “run the current program” – see red arrows in the screenshot below. The emulator will execute the code until it reaches the marked instruction and then halt, provided the program flow leads to that point. The content of registers and memory may also change based on the execution of instructions.

You may disregard any instructions and their use in the skeleton that lead up to your code as well as other advanced features in the RARS emulator. Their utility as well as bigger picture, including memory map, will be clear by the end of the assembly and software architecture material.



- 9) You can also run the program without any breakpoints. The epilog in the assembly skeleton file prints the two arguments and the final results to the console ("Run I/O" field) using system call assembly instruction "ecall" – see the screenshot below. In the case of implemented code ("add t0, t1, t2"), a2 register is not modified and is zero by the end of the program execution so 0 is printed to console.

CAUsers\johndoe\Desktop\lab2\mult.s - RARS 1.6

File Edit Run Settings Tools Help

Run speed at max (no interaction)

0101

Edit Execute

mult.s

```
92      la      a0, n1
93      ecall
94
95  # print integer
96      li      a7, 1
97      mv      a0, a1
98      ecall
99  # print string
100     li      a7, 4
101     la      a0, n1
102     ecall
103
104  # print integer
105     li      a7, 1
106     mv      a0, a2
107     ecall
108  # print string
109     li      a7, 4
110     la      a0, n1
111     ecall
112
113     jr      ra
114
115
```

Line: 99 Column: 1 ☒ Show Line Numbers

Messages Run I/O

Student
7
19
0

Clear

-- program is finished running (0) --

Registers Floating Point Control and Status

Name	Number	Value
zero	0	0x00000000
ra	1	0x00000000
sp	2	0x7ffffeffc
gp	3	0x10008000
tp	4	0x00000000
t0	5	0x00000007
t1	6	0x00000000
t2	7	0x00000001
s0	8	0x00000000
s1	9	0x00000000
a0	10	0x10010008
a1	11	0x00000013
a2	12	0x00000000
a3	13	0x00000000
a4	14	0x00000000
a5	15	0x00000000
a6	16	0x00000000
a7	17	0x0000000a
s2	18	0x00000000
s3	19	0x00000000
s4	20	0x00000000
s5	21	0x00000000
s6	22	0x00000000
s7	23	0x00000000
s8	24	0x00000000
s9	25	0x00000000
s10	26	0x00000000
s11	27	0x00000000
t3	28	0x00000000
t4	29	0x00000000
t5	30	0x00000000
t6	31	0x00000000
pc		0x0040005c